

Kubernetes Pod CrashLoopBackOff Recovery Runbook

Document ID: RB-K8S-001 | Version: 2.3 | Last Updated: 2024-11-15

Category	Kubernetes	Severity	P1 - Critical
Domain	Container Orchestration	Est. Duration	15-30 minutes
Owner	Platform Engineering	Oncall Escalation	#platform-oncall
Tags	kubernetes, pods, crashloop, containers, k8s		

Overview

A CrashLoopBackOff occurs when a Kubernetes pod repeatedly fails to start. Kubernetes attempts to restart the container, but it crashes again immediately. The interval between restarts increases exponentially (10s, 20s, 40s, 80s, 160s, 300s max). Common causes include application startup errors, missing environment variables, insufficient memory/CPU, misconfigured liveness probes, or dependency failures.

Prerequisites

- kubectl configured and connected to the target cluster
- Read access to the affected namespace
- Access to container registry for image verification
- Slack access for #platform-oncall escalation channel

Response Steps

Step 1: Identify the Affected Pod

List all pods in the namespace and filter for those in CrashLoopBackOff state. Note the pod name, restart count, and age for triage priority.

```
kubectl get pods -n --field-selector=status.phase=Running
kubectl get pods -n | grep CrashLoopBackOff
```

NOTE: High restart counts (>10) indicate a persistent configuration issue, not a transient failure.

Step 2: Inspect Pod Events and Status

Describe the pod to view events, exit codes, and last state. The exit code reveals the root cause: 1=application error, 137=OOMKilled, 139=segfault.

```
kubectl describe pod -n
kubectl get events -n --sort-by=.lastTimestamp | tail -20
```

Step 3: Retrieve Container Logs

Fetch logs from the current container and the previous crashed container to identify the startup error or exception.

```
kubectl logs -n --previous
kubectl logs -n -c --tail=100
```

Step 4: Check Resource Limits

OOMKilled (exit code 137) means the container exceeded its memory limit. Check current limits and node capacity before increasing.

```
kubectl top pods -n
kubectl top nodes
kubectl get pod -n -o jsonpath='{.spec.containers[*].resources}'
```

Step 5: Verify Environment Variables and Secrets

Missing or misconfigured environment variables are a top cause of CrashLoopBackOff. Verify all required secrets and configmaps exist in the namespace.

```
kubectl get secrets -n
kubectl get configmaps -n
kubectl describe deployment -n | grep -A20 'Environment'
```

Step 6: Scale Down and Redeploy

If a configuration change or image update is needed, scale the deployment to zero before applying changes. This prevents cascading restarts during the fix.

```
kubectl scale deployment --replicas=0 -n
kubectl set image deployment/ =: -n
kubectl scale deployment --replicas=3 -n
```

Step 7: Verify Recovery

Monitor the pod rollout to confirm all replicas reach Running state with zero restart count. Wait for the readiness probe to pass.

```
kubectl rollout status deployment/ -n
kubectl get pods -n -w
```

SUCCESS criteria: All pods show STATUS=Running, RESTARTS=0, and READY=1/1 (or N/N).

Rollback Steps

If the new deployment does not recover, roll back to the previous stable version.

Rollback Step 1: Undo Deployment

```
kubectl rollout undo deployment/ -n
```

Rollback Step 2: Verify Rollback

```
kubectl rollout status deployment/ -n
kubectl rollout history deployment/ -n
```

Rollback Step 3: Alert Platform Team

Post incident summary to Slack #platform-oncall with: pod name, namespace, root cause, timeline, and rollback status.

Escalation Path

Time Without Resolution	Action	Contact
0-15 min	Follow this runbook	On-call engineer
15-30 min	Escalate to platform lead	#platform-oncall Slack
30-60 min	Declare P1 incident	Engineering Manager + CTO
>60 min	Invoke DR procedure	VP Engineering

Step Dependencies

Steps must be executed in sequence. Step 6 (Scale Down and Redeploy) depends on Step 4 (Resource Check) and Step 5 (Env Var Verification) completing successfully. Rollback steps depend on Step 6 having been attempted first.

Step	Depends On	Can Parallelize With
Step 1: Identify Pod	—	—
Step 2: Inspect Events	Step 1	—
Step 3: Retrieve Logs	Step 1	Step 2
Step 4: Check Resources	Step 2, Step 3	Step 5
Step 5: Verify Env Vars	Step 2, Step 3	Step 4
Step 6: Scale and Redeploy	Step 4, Step 5	—
Step 7: Verify Recovery	Step 6	—

This runbook is maintained by the Platform Engineering team. For corrections or updates, open a PR to the runbooks repository or ping #platform-oncall.